

TABLE DES MATIÈRES

PARTIE I – STATISTIQUES ET PROBABILITÉS EN DATA SCIENCE
PARTIE II – ANALYSE DES SÉRIES TEMPORELLES
 2.1 Par les Moindres Carrés Ordinaires (MCO)
 2.2 Par le Lissage
PARTIE III – RÉDUCTION DE MATRICE ET DÉCOMPOSITION
PARTIE IV – ÉQUATIONS DIFFÉRENTIELLES EN DATA SCIENCE
PARTIE V – OPTIMISATION : EXTREMUM, GRADIENT ET HESSIEN

PARTIE I – STATISTIQUES ET PROBABILITÉS EN DATA SCIENCE

=====

1.1 POURQUOI LES STATISTIQUES SONT LE CŒUR DE LA DATA SCIENCE

=====

Un data scientist sans statistiques solides construit sur du sable.
Tout modèle, toute prédiction, toute décision repose sur des fondements
probabilistes et statistiques. Ignorer ces fondements, c'est produire des
résultats impossibles à interpréter, à défendre, et à fiabiliser.

VÉRITÉ FONDAMENTALE :

"Les données ne parlent pas d'elles-mêmes. Elles murmurent. La statistique
est l'art d'entendre ce murmure et de le traduire en certitude mesurée."

DISTINCTION ESSENTIELLE

- Statistique descriptive : décrire ce qui s'est passé
- Statistique inférentielle : généraliser à partir d'un échantillon
- Probabilité : modéliser l'incertitude future
- Statistique bayésienne : mettre à jour les croyances avec les preuves

1.2 STATISTIQUES DESCRIPTIVES – SOCLE OPÉRATIONNEL

=====

MESURES DE TENDANCE CENTRALE

[A] MOYENNE ARITHMÉTIQUE

$$\mu = (1/n) \times \sum x_i$$

Usage : variables symétriques sans outliers
Piège : sensible aux valeurs extrêmes
Exemple : salaire moyen faussé par un PDG à 10M€/an

[B] MÉDIANE

Valeur centrale qui divise la distribution en deux parties égales.

Usage : distributions asymétriques, revenus, temps de réponse
Avantage : robuste aux outliers
Règle : si moyenne >> médiane → distribution étalée à droite (right-skewed)

[C] MODE

Valeur la plus fréquente.

Usage : variables catégorielles, distributions multimodales
Exemple : taille de chaussure la plus vendue

RÈGLE PRATIQUE GCI :

Toujours regarder la DISTRIBUTION avant de calculer une moyenne.
Une moyenne sans distribution est une illusion de précision.
→ Tracer l'histogramme. Toujours.

MESURES DE DISPERSION

[A] VARIANCE

$$\sigma^2 = (1/n) \times \sum (x_i - \mu)^2$$

Unité au carré → difficile à interpréter directement.
Mais fondamentale pour l'algèbre linéaire et le ML.

[B] ÉCART-TYPE

$$\sigma = \sqrt{\sigma^2}$$

Même unité que la variable → interprétable directement.
"Les ventes varient en moyenne de ± 1 200€ autour de la moyenne."

[C] COEFFICIENT DE VARIATION (CV)

$$CV = (\sigma / \mu) \times 100$$

Usage : comparer la dispersion entre variables d'unités différentes.
CV < 15% → faible variabilité
CV > 30% → forte variabilité, modèle plus difficile à ajuster

[D] PERCENTILES ET QUARTILES

P25 = Q1 (premier quartile)

P50 = Q2 = Médiane

P75 = Q3 (troisième quartile)

IQR = Q3 - Q1 (écart interquartile)

Détection des outliers par méthode IQR :

Outlier si : $x < Q1 - 1.5 \times IQR$ ou $x > Q3 + 1.5 \times IQR$

[E] ASYMÉTRIE (SKEWNESS) ET APLATISSEMENT (KURTOSIS)

Skewness > 0 : queue à droite (revenus, prix immobilier)

Skewness < 0 : queue à gauche (notes d'étudiants bien préparés)

Skewness = 0 : distribution symétrique

Kurtosis > 3 (leptokurtique) : queues épaisses, événements extrêmes fréquents

Kurtosis < 3 (platykurtique) : queues fines, peu de valeurs extrêmes

Kurtosis = 3 : distribution normale (mésokurtique)

Application data science :

→ Kurtosis élevé dans des données financières = risque de tail event

→ Orienter le choix du modèle (normal vs robust vs heavy-tail)

1.3 PROBABILITÉS – MODÉLISER L'INCERTITUDE

AXIOMES DE BASE

$$P(A) \in [0, 1]$$

$$P(\Omega) = 1 \text{ (espace total)}$$

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

PROBABILITÉ CONDITIONNELLE

$$P(A|B) = P(A \cap B) / P(B)$$

Lecture : "Probabilité que A se réalise, sachant que B est déjà réalisé."

Exemple data science : $P(\text{churn} | \text{client_premium} = \text{True})$

THÉORÈME DE BAYES – LA RÈGLE D'OR DU DATA SCIENTIST

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

– P(A) : probabilité a priori (avant observation)

– P(B|A) : vraisemblance (likelihood)

– P(A|B) : probabilité a posteriori (après observation)

– P(B) : évidence (constante de normalisation)

APPLICATION CONCRÈTE – DÉTECTION DE FRAUDE :

$$P(\text{fraude}) = 0.01 \quad \leftarrow \text{rare (a priori)}$$

$$P(\text{alerte} | \text{fraude}) = 0.95 \quad \leftarrow \text{le modèle détecte bien}$$

$$P(\text{alerte} | \text{non-fraude}) = 0.10 \quad \leftarrow \text{faux positifs}$$

$$\begin{aligned} P(\text{fraude} | \text{alerte}) &= (0.95 \times 0.01) / [(0.95 \times 0.01) + (0.10 \times 0.99)] \\ &= 0.0095 / 0.1090 \end{aligned}$$

≈ 8.7%

INTERPRÉTATION :

Même avec un modèle à 95% de détection, une alerte n'a que 8.7% de chance d'être une vraie fraude si la fraude est rare.

→ Justifie l'importance du seuil de décision et du recall vs precision.

LES DISTRIBUTIONS DE PROBABILITÉ ESSENTIELLES

[1] LOI NORMALE (GAUSSIENNE)

$$f(x) = (1 / \sigma\sqrt{2\pi}) \times \exp(-(x-\mu)^2/2\sigma^2)$$

Paramètres : μ (moyenne), σ (écart-type)

Usage data science :

- Résidus d'une régression linéaire (hypothèse MC0)
- Bruit dans les capteurs IoT
- Scores standardisés
- Base du théorème central limite

Règle 68-95-99.7 :

68% des valeurs se trouvent dans $[\mu - \sigma ; \mu + \sigma]$

95% des valeurs se trouvent dans $[\mu - 2\sigma ; \mu + 2\sigma]$

99.7% des valeurs se trouvent dans $[\mu - 3\sigma ; \mu + 3\sigma]$

[2] LOI BINOMIALE

$$P(X = k) = C(n, k) \times p^k \times (1-p)^{(n-k)}$$

Paramètres : n (nombre d'essais), p (probabilité de succès)

Usage data science :

- Taux de clics (CTR) sur n impressions
- Tests A/B (k conversions sur n visiteurs)
- Contrôle qualité (k défauts sur n pièces)

[3] LOI DE POISSON

$$P(X = k) = (\lambda^k \times e^{(-\lambda)}) / k!$$

Paramètre : λ (nombre moyen d'événements par intervalle)

Usage data science :

- Nombre de requêtes serveur par seconde
- Nombre de pannes machine par mois
- Nombre d'emails reçus par heure

[4] LOI EXPONENTIELLE

$$f(x) = \lambda \times e^{(-\lambda x)} \quad \text{pour } x \geq 0$$

Usage data science :

- Temps entre deux événements (inter-arrivées)
- Durée de vie d'un composant
- Temps de réponse d'un service web

[5] LOI DE BERNOULLI

$$P(X = 1) = p, \quad P(X = 0) = 1 - p$$

Cas particulier de la binomiale avec $n = 1$.

Usage : classification binaire, conversion oui/non

[6] LOI BETA

Paramètres : α , β (forme)

Usage :

- Modéliser un taux ou une probabilité (valeur entre 0 et 1)
- Prior conjugué dans les modèles bayésiens de classification
- Tests A/B bayésiens : prior sur le taux de conversion

[7] LOI GAMMA

Usage :

- Modéliser des durées ou des montants positifs
- Généralisation de la loi exponentielle
- Modélisation de la latence dans les systèmes distribués

1.4 INFÉRENCE STATISTIQUE – GÉNÉRALISER À PARTIR DES DONNÉES

THÉORÈME CENTRAL LIMITE (TCL)

Quelle que soit la distribution d'une variable X , la moyenne d'un

échantillon de taille n suffisamment grand suit approximativement une loi normale.

$$\bar{X} \sim N(\mu, \sigma^2/n)$$

Seuil pratique : $n \geq 30$ (souvent suffisant)

Importance capitale : justifie l'utilisation de tests paramétriques même sur des données non-normales.

TESTS D'HYPOTHÈSES

STRUCTURE GÉNÉRALE :

H_0 : hypothèse nulle (statu quo, "pas d'effet")

H_1 : hypothèse alternative ("il y a un effet")

p-value : probabilité d'obtenir un résultat aussi extrême si H_0 est vraie

→ $p < \alpha$ (souvent 0.05) : on rejette H_0

→ $p \geq \alpha$: on ne peut pas rejeter H_0 ($\neq$ prouver H_0)

ERREURS À DISTINGUER :

Erreur de type I (α) : rejeter H_0 alors qu'elle est vraie (faux positif)

Erreur de type II (β) : ne pas rejeter H_0 alors qu'elle est fautive (faux négatif)

Puissance = $1 - \beta$: capacité à détecter un vrai effet

TESTS COURANTS ET LEURS USAGES DATA SCIENCE :

t-test à un échantillon

→ Comparer une moyenne à une valeur de référence

→ Exemple : "Le temps de traitement moyen est-il < 2 secondes ?"

t-test à deux échantillons indépendants

→ Comparer deux moyennes (groupes A vs B)

→ Exemple : "Les ventes du groupe test $>$ groupe contrôle ?"

t-test apparié (Paired t-test)

→ Mêmes sujets, deux conditions (avant/après)

→ Exemple : "Le modèle améliore-t-il le score après déploiement ?"

ANOVA (Analysis of Variance)

→ Comparer plus de deux groupes simultanément

→ Exemple : "Les 4 régions ont-elles des ventes moyennes différentes ?"

→ Extension : ANOVA à deux facteurs pour interactions

Chi-carré d'indépendance

→ Tester l'indépendance entre deux variables catégorielles

→ Exemple : "Le genre influence-t-il le choix du produit ?"

Test de Kolmogorov-Smirnov

→ Comparer deux distributions empiriques

→ Usage : détecter le data drift (dérive de la distribution en prod)

Mann-Whitney U (non-paramétrique)

→ Alternative au t-test si distribution non-normale et n petit

→ Robuste mais moins puissant

INTERVALLES DE CONFIANCE

IC à $(1-\alpha)\%$ pour μ :

$$[\bar{X} - z_{\alpha/2} \times (\sigma/\sqrt{n}) ; \bar{X} + z_{\alpha/2} \times (\sigma/\sqrt{n})]$$

Pour $\alpha = 0.05$: $z = 1.96$

Pour $\alpha = 0.01$: $z = 2.576$

Interprétation correcte :

"Si on répétait l'expérience 100 fois, l'IC contiendrait le vrai paramètre dans 95% des cas."

($\neq$ "Il y a 95% de chances que μ soit dans cet intervalle")

Application data science :

– Intervalle de confiance sur les métriques d'un modèle

– Borne de l'erreur de prédiction

– Validation des résultats de tests A/B

1.5 CORRÉLATION, RÉGRESSION ET CAUSALITÉ

CORRÉLATION DE PEARSON

$$r = \frac{\sum[(x_i - \bar{x})(y_i - \bar{y})]}{[\sqrt{\sum(x_i - \bar{x})^2} \times \sqrt{\sum(y_i - \bar{y})^2}]}$$

$$r \in [-1, 1]$$

$r = +1$: parfaite corrélation positive

$r = 0$: aucune corrélation linéaire

$r = -1$: parfaite corrélation négative

Règle empirique :

$|r| < 0.3 \rightarrow$ faible

$|r| \in [0.3, 0.7] \rightarrow$ modérée

$|r| > 0.7 \rightarrow$ forte

CORRÉLATION DE SPEARMAN

Basée sur les rangs, robuste aux outliers et non-linéarités.

Usage : données ordinales ou distributions non normales.

PIÈGE FONDAMENTAL :

"Corrélation $\neq$ Causalité"

Exemple classique : la corrélation entre ventes de glaces et noyades est forte. La cause commune est la chaleur estivale, pas les glaces.

$\rightarrow$ Toujours chercher les variables confondantes (confounders).

RÉGRESSION LINÉAIRE SIMPLE

$$Y = \beta_0 + \beta_1 \cdot X + \varepsilon$$

β_1 : variation de Y pour une augmentation unitaire de X

β_0 : valeur de Y quand $X = 0$ (ordonnée à l'origine)

ε : terme d'erreur (résidu)

R^2 (coefficient de détermination) :

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

$R^2 \in [0, 1]$: proportion de variance de Y expliquée par X

RÉGRESSION LINÉAIRE MULTIPLE

$$Y = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \dots + \beta_p \cdot X_p + \varepsilon$$

$\rightarrow$ Estimée par les Moindres Carrés Ordinaires (voir Partie II)

$\rightarrow R^2$ ajusté : pénalise l'ajout de variables inutiles

$\rightarrow$ VIF (Variance Inflation Factor) : détecter la multicolinéarité

PARTIE II – ANALYSE DES SÉRIES TEMPORELLES

2.0 INTRODUCTION AUX SÉRIES TEMPORELLES

UNE SÉRIE TEMPORELLE EST :

Une suite d'observations ordonnées dans le temps :

$$\{y_t\} = \{y_1, y_2, \dots, y_T\} \text{ avec } t = 1, 2, \dots, T$$

L'ordre temporel est fondamental. Contrairement aux données cross-sectionnelles, les observations ne sont PAS indépendantes – chaque valeur dépend (potentiellement) de ses valeurs passées.

DÉCOMPOSITION CLASSIQUE D'UNE SÉRIE :

$$Y_t = T_t + S_t + C_t + \varepsilon_t$$

– T_t : tendance (trend) – évolution de long terme

– S_t : saisonnalité – pattern régulier (annuel, mensuel, hebdomadaire)

– C_t : cycle – fluctuation à long terme (cycles économiques)

– ε_t : résidu / bruit (composante aléatoire)

Modèle additif : $Y_t = T_t + S_t + \varepsilon_t$ (amplitudes constantes)

Modèle multiplicatif : $Y_t = T_t \times S_t \times \varepsilon_t$ (amplitudes proportionnelles à T_t)

STATIONNARITÉ – CONCEPT CLEF

Une série est stationnaire si :

– $E[Y_t] = \mu$ (moyenne constante dans le temps)

– $\text{Var}[Y_t] = \sigma^2$ (variance constante dans le temps)

– $\text{Cov}(Y_t, Y_{t+k})$ dépend de k mais pas de t

Test de stationnarité :

Test ADF (Augmented Dickey-Fuller)

H_0 : la série est non-stationnaire (racine unitaire)

Si $p < 0.05 \rightarrow$ rejeter $H_0 \rightarrow$ série stationnaire

Rendre une série stationnaire :

– Différenciation : $\Delta Y_t = Y_t - Y_{t-1}$

– Transformation logarithmique : $Z_t = \ln(Y_t)$

– Différenciation saisonnière : $\Delta_s Y_t = Y_t - Y_{t-s}$

2.1 ANALYSE PAR LES MOINDRES CARRÉS ORDINAIRES (MCO)

PRINCIPE DES MOINDRES CARRÉS ORDINAIRES

L'objectif est de trouver les coefficients β qui minimisent la somme des carrés des résidus (erreurs) entre les valeurs observées et ajustées :

Minimiser : $S(\beta) = \sum \varepsilon_t^2 = \sum (y_t - \hat{y}_t)^2$ pour $t = 1, \dots, T$

En notation matricielle :

$S(\beta) = (Y - X\beta)^T(Y - X\beta)$

SOLUTION ANALYTIQUE (estimateur OLS) :

$\hat{\beta} = (X^T X)^{-1} X^T Y$

Conditions d'existence : $(X^T X)$ doit être inversible $\rightarrow$ pas de multicollinéarité parfaite.

HYPOTHÈSES DE GAUSS-MARKOV (pour que MCO soit BLUE)

Best Linear Unbiased Estimator

[H1] Linéarité : $E[y_t] = X_t \beta$ (le modèle est bien linéaire)

[H2] Exogénéité : $E[\varepsilon_t | X] = 0$ (résidus non corrélés aux régresseurs)

[H3] Sphéricité : $\text{Var}[\varepsilon | X] = \sigma^2 I$ (homoscédasticité + absence d'autocorrélation)

[H4] Rang plein : $\text{rang}(X) = p$ (pas de multicollinéarité parfaite)

[H5] Normalité : $\varepsilon \sim N(0, \sigma^2 I)$ (pour l'inférence exacte)

VIOLATION COURANTE EN SÉRIE TEMPORELLE : [H3]

Les résidus sont souvent autocorrélés dans le temps $\rightarrow$ MCO reste non-biaisé mais perd son efficacité (les écarts-types sont mal estimés).

RÉGRESSION TEMPORELLE SIMPLE PAR MCO

MODÈLE DE TENDANCE LINÉAIRE :

$y_t = \beta_0 + \beta_1 \cdot t + \varepsilon_t$

Ici, le régresseur est le temps $t = 1, 2, \dots, T$.

Exemple : modéliser la croissance linéaire des ventes dans le temps.

$\hat{\beta}_1 > 0 \rightarrow$ tendance haussière

$\hat{\beta}_1 < 0 \rightarrow$ tendance baissière

$\hat{\beta}_1 \approx 0 \rightarrow$ pas de tendance linéaire claire

MODÈLE DE TENDANCE POLYNOMIALE :

$y_t = \beta_0 + \beta_1 \cdot t + \beta_2 \cdot t^2 + \varepsilon_t$

Usage : capturer une accélération ou un ralentissement de la tendance.

MODÈLE MCO AVEC VARIABLES INDICATRICES (DUMMY) POUR LA SAISONNALITÉ

Exemple avec données trimestrielles (4 saisons) :

$y_t = \beta_0 + \beta_1 \cdot t + \delta_1 \cdot D1t + \delta_2 \cdot D2t + \delta_3 \cdot D3t + \varepsilon_t$

$Dkt = 1$ si t est dans le trimestre k , 0 sinon

On omet une saison de référence (ici T4) pour éviter la multicollinéarité parfaite.

δk : l'effet saisonnier du trimestre k par rapport à la référence T4.

Après estimation, on peut extraire :

– La tendance : $\hat{\beta}_0 + \hat{\beta}_1 \cdot t$

– L'effet saisonnier : $\hat{\delta}^k$

- Le résidu désaisonnalisé : ε_t

MCO AVEC AUTOCORRÉLATION – CORRECTION

En série temporelle, les résidus ε_t sont souvent corrélés :

$$\varepsilon_t = \rho \cdot \varepsilon_{t-1} + u_t \quad (\text{processus AR}(1))$$

Conséquence : les tests t et F deviennent invalides.

TEST D'AUTOCORRÉLATION :

- Durbin-Watson : $d \approx 2 \rightarrow$ pas d'autocorrélation
- $d < 2 \rightarrow$ autocorrélation positive
- $d > 2 \rightarrow$ autocorrélation négative

Breusch-Godfrey : test général d'autocorrélation d'ordre q

CORRECTIONS :

- GLS (Generalized Least Squares) avec structure d'autocorrélation connue
- FGLS (Feasible GLS) : estimer ρ d'abord, puis corriger
- Newey-West : correction des écarts-types (HAC - Heteroskedasticity and Autocorrelation Consistent), sans modifier les coefficients

MCO APPLIQUÉ À LA RÉGRESSION AUTOREGRESSIVE

Modèle AR(p) estimé par MCO :

$$y_t = \phi_1 \cdot y_{t-1} + \phi_2 \cdot y_{t-2} + \dots + \phi_p \cdot y_{t-p} + \varepsilon_t$$

- $\rightarrow$ OLS régresse y_t sur ses propres valeurs passées
- $\rightarrow$ Choix de p : critère AIC / BIC (minimiser la pénalité)

$$AIC = 2k - 2\ln(L)$$

$$BIC = k \cdot \ln(n) - 2\ln(L)$$

k = nombre de paramètres, L = vraisemblance maximale

$\rightarrow$ Choisir le modèle qui minimise AIC ou BIC.

MODÈLE ARIMA PAR MCO ÉTENDU

ARIMA(p, d, q) :

- p : ordre autorégressif (termes AR)
- d : ordre de différenciation (pour stationnariser)
- q : ordre de moyenne mobile (termes MA)

ÉTAPES :

- [1] Différencier d fois pour stationnariser (ADF)
- [2] Identifier p et q via ACF (autocorrélation) et PACF (autocorrélation partielle)
- [3] Estimer par MLE (équivalent MCO pour les composantes MA)
- [4] Valider les résidus (bruit blanc)
- [5] Prédire hors-échantillon

SARIMA(p,d,q)(P,D,Q)s :

Extension saisonnière avec paramètres supplémentaires (P, D, Q) et période s.

Exemple : SARIMA(1,1,1)(1,1,1)12 pour données mensuelles avec saisonnalité annuelle.

VALIDATION DU MODÈLE MCO EN SÉRIES TEMPORELLES

Validation hors-échantillon obligatoire :

- $\rightarrow$ Ne jamais utiliser les données futures pour entraîner le modèle
- $\rightarrow$ Utiliser un split temporel : 80% entraînement / 20% test (chronologique)
- $\rightarrow$ Jamais un split aléatoire (violerait la structure temporelle)

Métriques d'évaluation :

$$\begin{aligned} MAE &= (1/T) \times \sum |y_t - \hat{y}_t| && (\text{erreur absolue moyenne}) \\ RMSE &= \sqrt{(1/T) \times \sum (y_t - \hat{y}_t)^2} && (\text{erreur quadratique moyenne}) \\ MAPE &= (100/T) \times \sum |y_t - \hat{y}_t| / |y_t| && (\text{erreur absolue en \%}) \end{aligned}$$

2.2 ANALYSE PAR LE LISSAGE (SMOOTHING)

PRINCIPE DU LISSAGE

Éliminer le bruit d'une série temporelle pour révéler sa structure sous-jacente (tendance, saisonnalité) sans poser d'hypothèses fortes sur le processus générateur des données.

Philosophie : "moins de paramètres, plus de robustesse."

Adapté aux prévisions opérationnelles à court terme.

LISSAGE PAR MOYENNE MOBILE (Moving Average)

MOYENNE MOBILE SIMPLE (MMS) d'ordre k :

$$Mat = (1/k) \times \sum y_{t-i} \text{ pour } i = 0, \dots, k-1$$

Ou centrée sur une fenêtre symétrique :

$$Mat = (1/(2k+1)) \times \sum y_{t+i} \text{ pour } i = -k, \dots, k$$

- k petit : réactif aux changements, bruit résiduel élevé
- k grand : lisse mais réagit lentement aux ruptures

Usage data science :

- Détecter la tendance dans une série bruyante
- Visualisation et communication
- Prétraitement avant modélisation

MOYENNE MOBILE PONDÉRÉE :

$$MA_{pond_t} = \sum w_i \times y_{t-i} \text{ avec } \sum w_i = 1$$

→ Donner plus de poids aux observations récentes

LISSAGE EXPONENTIEL SIMPLE (LES)

$$\hat{Y}_{t+1} = \alpha \cdot y_t + (1-\alpha) \cdot \hat{Y}_t$$

Ou de façon équivalente :

$$\hat{Y}_{t+1} = \hat{Y}_t + \alpha \cdot (y_t - \hat{Y}_t) = \hat{Y}_t + \alpha \cdot e_t$$

$\alpha \in (0, 1)$: paramètre de lissage

- α proche de 1 : forte réactivité, peu de lissage
- α proche de 0 : lissage fort, série très amortie

Développement infini :

$$\hat{Y}_{t+1} = \alpha \cdot y_t + \alpha(1-\alpha) \cdot y_{t-1} + \alpha(1-\alpha)^2 \cdot y_{t-2} + \dots$$

→ Toutes les valeurs passées contribuent, avec un poids décroissant exponentiellement vers le passé.

Usage : séries sans tendance ni saisonnalité marquées.

Optimisation de α : minimiser la SSE = $\sum (y_t - \hat{Y}_t)^2$

LISSAGE EXPONENTIEL DOUBLE – MÉTHODE DE HOLT

Pour les séries avec une tendance linéaire mais sans saisonnalité.

$$\text{Niveau : } L_t = \alpha \cdot y_t + (1-\alpha) \cdot (L_{t-1} + b_{t-1})$$

$$\text{Tendance: } b_t = \beta \cdot (L_t - L_{t-1}) + (1-\beta) \cdot b_{t-1}$$

Prévision à h pas :

$$\hat{Y}_{t+h} = L_t + h \cdot b_t$$

$\alpha \in (0,1)$: paramètre de lissage du niveau

$\beta \in (0,1)$: paramètre de lissage de la tendance

→ Deux équations permettent de séparer et extrapoler le niveau et la pente.

LISSAGE EXPONENTIEL TRIPLE – MÉTHODE DE HOLT-WINTERS

Pour les séries avec tendance ET saisonnalité.

VERSION ADDITIVE (amplitudes saisonnières constantes) :

$$\text{Niveau : } L_t = \alpha \cdot (y_t - S_{t-s}) + (1-\alpha) \cdot (L_{t-1} + b_{t-1})$$

$$\text{Tendance: } b_t = \beta \cdot (L_t - L_{t-1}) + (1-\beta) \cdot b_{t-1}$$

$$\text{Saison : } S_t = \gamma \cdot (y_t - L_t) + (1-\gamma) \cdot S_{t-s}$$

$$\text{Prévision : } \hat{Y}_{t+h} = L_t + h \cdot b_t + S_{t-s+h}$$

VERSION MULTIPLICATIVE (amplitudes saisonnières proportionnelles au niveau) :

$$\text{Niveau : } L_t = \alpha \cdot (y_t / S_{t-s}) + (1-\alpha) \cdot (L_{t-1} + b_{t-1})$$

$$\text{Tendance: } b_t = \beta \cdot (L_t - L_{t-1}) + (1-\beta) \cdot b_{t-1}$$

$$\text{Saison : } S_t = \gamma \cdot (y_t / L_t) + (1-\gamma) \cdot S_{t-s}$$

$$\text{Prévision : } \hat{Y}_{t+h} = (L_t + h \cdot b_t) \times S_{t-s+h}$$

s = période saisonnière (s=12 pour mensuel, s=4 pour trimestriel)
 α, β, γ : optimisés par minimisation de la MSE ou MLE

AVANTAGE DE HOLT-WINTERS :

- Simple à implémenter et à interpréter
- Très efficace pour des prévisions à court et moyen terme
- Adaptatif : se recalibrer au fil des nouvelles données

LISSAGE PAR NOYAU (KERNEL SMOOTHING)

$$\hat{Y}(x) = \frac{\sum K[(x - x_i)/h] \times y_i}{\sum K[(x - x_i)/h]}$$

$K(\cdot)$: fonction noyau (Gaussienne, Epanechnikov, etc.)

h : bande passante (bandwidth) – contrôle le degré de lissage

- Noyau Gaussien : $K(u) = (1/\sqrt{2\pi}) \times \exp(-u^2/2)$
- Epanechnikov : $K(u) = (3/4)(1-u^2)$ pour $|u| \leq 1$

Choix de h :

- h petit → sous-lissage (modèle complexe, overfit)
- h grand → sur-lissage (modèle trop simple, underfit)
- Optimal via cross-validation (leave-one-out)

LISSAGE LOESS (Locally Weighted Scatterplot Smoothing)

Ajustement local de polynômes pondérés par la distance.
 À chaque point x_0 , on ajuste une régression polynomiale locale en pondérant les observations voisines par une fonction noyau.

Avantages :

- Capte les non-linéarités locales
- Robuste aux outliers (version robust LOESS)
- Aucune hypothèse sur la forme globale de la courbe

Paramètre : span (α) = fraction des données utilisées localement

- span proche de 0 : très local, réactif, bruyant
- span proche de 1 : très global, lisse, proche d'une régression globale

Application data science :

- Décomposition STL (Seasonal and Trend decomposition using Loess)
- Visualisation de tendances dans les dashboards analytiques

COMPARAISON MCO VS LISSAGE

CRITÈRE	MCO	LISSAGE
Hypothèses	Nombreuses (Gauss-Markov)	Peu ou aucune
Interprétabilité	Forte (coefficients)	Faible (courbe lissée)
Inférence formelle	Oui (tests, IC)	Limitée
Prévision	Oui (extrapolation)	Court terme principalement
Réactivité	Fixe (re-estimer)	Adaptative (exponentiel)
Données requises	Minimum 30-50 obs.	Fonctionne avec peu de données
Gestion saisonnalité	Variables dummy / SARIMA	Holt-Winters directement

PARTIE III – RÉDUCTION DE MATRICE ET DÉCOMPOSITION

3.1 POURQUOI RÉDUIRE LA DIMENSIONNALITÉ

MALÉDICTION DE LA DIMENSIONNALITÉ (CURSE OF DIMENSIONALITY)

Plus le nombre de dimensions p augmente, plus l'espace devient vide.
 La densité des données décroît exponentiellement.
 Les distances perdent leur sens (tout devient équidistant).
 Les modèles surapprennent facilement.

Règle empirique : $n \gg p$ (besoin de beaucoup plus d'observations que de variables)
 → Quand p est grand (images, texte, génomique) → réduction obligatoire.

OBJECTIFS DE LA RÉDUCTION DIMENSIONNELLE

- Compresser l'information sans perte significative
- Visualiser des données en haute dimension (→ 2D/3D)

- Accélérer l'entraînement des modèles
- Réduire le bruit dans les données
- Traiter la multicolinéarité avant MCO ou régression

3.2 ALGÈBRE LINÉAIRE – FONDATIONS

VALEURS PROPRES ET VECTEURS PROPRES

Définition :

Pour une matrice carrée A de taille $n \times n$:

$$A \cdot v = \lambda \cdot v$$

v : vecteur propre (direction invariante par transformation)

λ : valeur propre correspondante (facteur d'étirement)

Calcul :

$$\det(A - \lambda I) = 0 \leftarrow \text{équation caractéristique}$$

→ Résoudre pour λ → trouver les p valeurs propres

→ Pour chaque λ , résoudre $(A - \lambda I)v = 0$ → trouver les vecteurs propres

Propriétés importantes :

- $\text{Trace}(A) = \sum \lambda_i$ (somme des valeurs propres = trace de la matrice)
- $\det(A) = \prod \lambda_i$ (produit des valeurs propres = déterminant)
- Si tous les $\lambda_i > 0$ → matrice définie positive (important pour Hessien)
- Les vecteurs propres d'une matrice symétrique sont orthogonaux

DÉCOMPOSITION SPECTRALE (EIGEN DECOMPOSITION)

Pour une matrice symétrique A (ex: matrice de covariance) :

$$A = Q \Lambda Q^T$$

Q : matrice des vecteurs propres (colonnes orthonormées)

Λ : matrice diagonale des valeurs propres ($\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$)

Usage data science :

- Fondation de PCA
- Analyse spectrale des graphes
- Systèmes de recommandation

3.3 ANALYSE EN COMPOSANTES PRINCIPALES (ACP / PCA)

INTUITION

PCA cherche de nouvelles axes de représentation (composantes principales) qui maximisent la variance expliquée, et qui sont orthogonaux entre eux.

→ Projeter les données sur un sous-espace de dimension $k \ll p$.

ALGORITHME PCA (ÉTAPE PAR ÉTAPE)

[1] CENTRER LES DONNÉES

$$X_{\text{centered}} = X - \text{mean}(X) \quad (\text{colonne par colonne})$$

(et standardiser si les variables ont des unités différentes)

[2] CALCULER LA MATRICE DE COVARIANCE

$$C = (1/(n-1)) \times X^T \cdot X \quad \text{de taille } p \times p$$

[3] DÉCOMPOSITION SPECTRALE DE C

$$C = V \cdot \Lambda \cdot V^T$$

V : matrice des vecteurs propres (composantes principales)

Λ : valeurs propres $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$

[4] CHOISIR k COMPOSANTES

Critère de variance expliquée :

$$\% \text{ var. expliquée par } k \text{ composantes} = \frac{\sum(\lambda_1 \dots \lambda_k)}{\sum(\lambda_1 \dots \lambda_p)}$$

→ Choisir k tel que 80-95% de la variance est conservée

Scree plot : graphique des λ_i triées → chercher le "coude"

[5] PROJETER LES DONNÉES

$$Z = X_{\text{centered}} \cdot V_k \quad (n \times k)$$

Z est le dataset réduit dans le nouvel espace des composantes.

VARIANCE EXPLIQUÉE ET INTERPRÉTATION

Composante principale 1 (PC1) :

→ Capture la direction de variance maximale dans l'espace original

Composante principale 2 (PC2) :

→ Orthogonale à PC1, capture la direction de variance maximale résiduelle

"Loadings" (charges) : coefficients des vecteurs propres

→ Indiquent comment chaque variable originale contribue à chaque CP

→ Permettent une interprétation métier des composantes

LIMITATIONS ET PRÉCAUTIONS

- PCA est linéaire → ne capture pas les structures non-linéaires
- Les composantes perdent l'interprétabilité des variables originales
- Sensible à l'échelle → standardiser impérativement avant PCA
- Inadapté si les variables sont catégorielles (→ MCA à la place)

APPLICATION PRATIQUE – PIPELINE ML

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
```

```
pca = PCA(n_components=0.95) # conserver 95% de variance
X_pca = pca.fit_transform(X_scaled)
# pca.explained_variance_ratio_ → part de variance par composante
```

3.4 DÉCOMPOSITION EN VALEURS SINGULIÈRES (SVD)

DÉFINITION

Toute matrice rectangulaire A de taille $m \times n$ peut s'écrire :

$$A = U \cdot \Sigma \cdot V^T$$

U : matrice $m \times m$ des vecteurs singuliers gauches (colonnes orthonormées)

Σ : matrice $m \times n$ diagonale des valeurs singulières $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$

V^T : matrice $n \times n$ des vecteurs singuliers droits (lignes orthonormées)

RELATION AVEC PCA

Si A est la matrice de données centrée :

- Les vecteurs propres de $A^T A = \text{colonnes de } V$ (composantes de PCA)
- Les valeurs propres de $A^T A = \sigma_i^2$ (carrés des valeurs singulières)
- SVD est l'algorithme numérique de référence pour calculer PCA

APPROXIMATION DE RANG RÉDUIT

On tronque en ne gardant que les k plus grandes valeurs singulières :

$$A \approx U_k \cdot \Sigma_k \cdot V_k^T$$

C'est la meilleure approximation de rang k de A au sens de Frobenius.

- Compression d'images
- Filtrage de bruit
- Systèmes de recommandation (matrix factorization)

APPLICATIONS DATA SCIENCE

[A] SYSTÈMES DE RECOMMANDATION (Collaborative Filtering)

Matrice d'interactions utilisateur $\times$ produit :

$$R \approx U \cdot \Sigma \cdot V^T$$

- Prédire les valeurs manquantes (notes non données)
- Netflix Prize : SVD tronquée a été la clé du modèle gagnant

[B] NLP – ANALYSE SÉMANTIQUE LATENTE (LSA)

Matrice termes $\times$ documents → SVD → réduction

- Capturer des similarités sémantiques cachées entre documents

[C] COMPRESSION D'IMAGES

Image en niveaux de gris = matrice de pixels A ($m \times n$)

- Garder $k = 20$ valeurs singulières sur 500
- Compression $\approx 96\%$, qualité visuelle préservée

[D] DÉTECTION D'ANOMALIES

- Reconstruire A avec SVD tronquée
- Erreur de reconstruction élevée → observation anormale

3.5 AUTRES MÉTHODES DE RÉDUCTION

t-SNE (t-Distributed Stochastic Neighbor Embedding)

- Non-linéaire, préserve les structures locales
- Idéal pour la visualisation 2D/3D
- Non déterministe (résultats varient d'une exécution à l'autre)
- Paramètre clé : perplexité (5-50 en pratique)
- ATTENTION : t-SNE n'est pas fait pour la prédiction, seulement la visualisation

UMAP (Uniform Manifold Approximation and Projection)

- Plus rapide que t-SNE
- Préserve mieux la structure globale
- Peut être utilisé pour réduction et visualisation
- Paramètres : n_neighbors, min_dist

NMF (Non-negative Matrix Factorization)

- $A \approx W \cdot H$ avec $W \geq 0$, $H \geq 0$
- Parties additives → meilleure interprétabilité
- Usage : analyse de topics (NLP), séparation de sources audio

PARTIE IV – ÉQUATIONS DIFFÉRENTIELLES EN DATA SCIENCE

4.1 POURQUOI LES ÉQUATIONS DIFFÉRENTIELLES EN DATA SCIENCE

Les équations différentielles (ED) modélisent des systèmes dynamiques : phénomènes dont l'état évolue continuellement dans le temps ou l'espace selon des lois de variation. En data science, elles permettent :

- D'injecter de la connaissance physique dans les modèles (Physics-Informed ML)
- De modéliser la diffusion d'informations (épidémies, réseaux sociaux)
- De décrire les dynamiques derrière les séries temporelles
- De comprendre l'algorithme de descente de gradient (ED continue)
- De résoudre des problèmes d'optimisation en temps continu

4.2 ÉQUATIONS DIFFÉRENTIELLES ORDINAIRES (EDO)

DÉFINITION

Une EDO d'ordre n exprime une relation entre une fonction $f(t)$ et ses dérivées jusqu'à l'ordre n :

$$F(t, y, y', y'', \dots, y^{(n)}) = 0$$

Où $y = y(t)$ est la fonction inconnue.

EDO D'ORDRE 1 – FORME CANONIQUE

$$dy/dt = f(t, y)$$

- Linéaire : $f(t, y) = a(t) \cdot y + b(t)$
- Non-linéaire : $f(t, y)$ est une fonction non-linéaire de y

Solution générale : $y(t) = y_{\text{particulière}} + y_{\text{homogène}} + C$ (constante d'intégration)

→ Condition initiale $y(t_0) = y_0$ pour déterminer C

EXEMPLES FONDAMENTAUX

[A] CROISSANCE EXPONENTIELLE / DÉCROISSANCE

$$dy/dt = k \cdot y$$

$$\text{Solution : } y(t) = y_0 \cdot e^{(kt)}$$

- $k > 0$: croissance (population bactérienne, intérêts composés)
- $k < 0$: décroissance (désintégration radioactive, amortissement)

Application data science :

- Modéliser la décroissance de la valeur d'un actif

– Modéliser la diffusion initiale d'un virus ou d'un produit viral

[B] MODÈLE LOGISTIQUE (CROISSANCE LIMITÉE)

$$dy/dt = k \cdot y \cdot (1 - y/K)$$

K : capacité de charge (population maximale)

$$\text{Solution : } y(t) = K / [1 + ((K - y_0)/y_0) \cdot e^{(-kt)}]$$

Application data science :

- Adoption de produits technologiques (courbe en S)
- Modèles épidémiologiques (fraction de population infectée)
- Fonction sigmoïde en ML = cas particulier (K=1, $y_0=0.5$)

[C] MODÈLE SIR ÉPIDÉMIOLOGIQUE

$$dS/dt = -\beta \cdot S \cdot I$$

$$dI/dt = \beta \cdot S \cdot I - \gamma \cdot I$$

$$dR/dt = \gamma \cdot I$$

S : susceptibles, I : infectés, R : rétablis

β : taux de transmission, γ : taux de guérison

$R_0 = \beta/\gamma$: nombre de reproduction de base

Application data science :

- Modélisation COVID-19 (prévisions hospitalières)
- Propagation virale sur les réseaux sociaux
- Diffusion de l'information / rumeurs

[D] OSCILLATEUR HARMONIQUE AMORTI

$$d^2y/dt^2 + 2\zeta\omega_0 \cdot dy/dt + \omega_0^2 \cdot y = 0$$

ζ : coefficient d'amortissement

ω_0 : fréquence propre

Application data science :

- Modéliser les cycles économiques (boom/bust)
- Analyse vibratoire dans la maintenance prédictive
- Modélisation de la réponse de systèmes de contrôle

4.3 ÉQUATIONS DIFFÉRENTIELLES ET DESCENTE DE GRADIENT

GRADIENT FLOW – L'EDO DERRIÈRE L'OPTIMISATION

La descente de gradient à pas infinitésimal s'écrit comme une EDO :

$$d\theta/dt = -\nabla L(\theta)$$

$\theta(t)$: paramètres du modèle à l'instant t

$L(\theta)$: fonction de perte

→ Le paramètre évolue continuellement dans la direction opposée au gradient.

→ La solution $\theta(t)$ converge vers un minimum local (si L est convexe : minimum global).

Version discrète (descente de gradient usuelle) :

$$\theta_{k+1} = \theta_k - \eta \cdot \nabla L(\theta_k)$$

est une approximation d'Euler de l'EDO :

$$\theta_{k+1} \approx \theta_k + \Delta t \cdot (-\nabla L(\theta_k)) \quad \text{avec } \eta = \Delta t$$

Importance : comprendre le gradient flow permet d'analyser la convergence, la stabilité, et le choix du pas η (learning rate).

NEURAL ODE – LES RÉSEAUX DE NEURONES COMME EDO

Concept moderne (Chen et al., 2018) :

Paramétrer la dérivée d'un état caché par un réseau de neurones :

$$dh(t)/dt = f_\theta(h(t), t)$$

Avec condition initiale $h(t_0) = x$ (entrée du réseau).

La sortie est $h(T) = \text{solveur_EDO}(f_\theta, x, t_0 \rightarrow T)$.

Avantages :

- Profondeur continue → mémoire constante
- Naturellement adapté aux séries temporelles irrégulières
- Interpolation entre observations non uniformément espacées

4.4 RÉSOLUTION NUMÉRIQUE DES EDO

Quand la solution analytique n'existe pas → résolution numérique.

MÉTHODE D'EULER (ordre 1)

$$y_{n+1} = y_n + h \cdot f(t_n, y_n)$$

h : pas de temps

Simple mais imprécis pour les systèmes raides (stiff ODEs).

MÉTHODE DE RUNGE-KUTTA D'ORDRE 4 (RK4)

$$\begin{aligned} k_1 &= h \cdot f(t_n, y_n) \\ k_2 &= h \cdot f(t_n + h/2, y_n + k_1/2) \\ k_3 &= h \cdot f(t_n + h/2, y_n + k_2/2) \\ k_4 &= h \cdot f(t_n + h, y_n + k_3) \\ y_{n+1} &= y_n + (k_1 + 2k_2 + 2k_3 + k_4) / 6 \end{aligned}$$

Précision d'ordre 4 → erreur locale en $O(h^5)$.

Standard de facto pour les EDO non raides.

IMPLÉMENTATION PYTHON

```
from scipy.integrate import solve_ivp

def model(t, y, k):
    return -k * y

sol = solve_ivp(model, t_span=[0, 10], y0=[1.0], args=(0.5,),
                method='RK45', dense_output=True)
```

PARTIE V – OPTIMISATION : EXTREMUM, GRADIENT ET HESSIEN

5.1 FONDEMENTS DE L'OPTIMISATION EN DATA SCIENCE

DÉFINITION DU PROBLÈME D'OPTIMISATION

Trouver θ^* tel que :

$$\theta^* = \operatorname{argmin}_{\{\theta \in \Theta\}} L(\theta)$$

$L(\theta)$: fonction objectif (loss function, fonction de perte)

Θ : espace des paramètres

Tout entraînement de modèle ML = résolution d'un problème d'optimisation.

- Régression linéaire : minimiser $\sum (y_i - \hat{y}_i)^2$
- Réseau de neurones : minimiser l'entropie croisée
- SVM : maximiser la marge de séparation

CONVEXITÉ – LE CONCEPT QUI CHANGE TOUT

Fonction convexe :

$$f[\lambda x + (1-\lambda)y] \leq \lambda f(x) + (1-\lambda)f(y) \quad \text{pour tout } \lambda \in [0,1]$$

- Un minimum local d'une fonction convexe est le minimum global.
- L'optimisation est "facile" sur une fonction convexe.

Non-convexe (réseaux de neurones) :

- Minima locaux multiples
- Selles (saddle points) nombreuses
- Convergence vers un minimum local, pas nécessairement global

Heureusement : en pratique, les minima locaux des grands réseaux de neurones ont souvent une valeur très proche du minimum global (résultat empirique et théorique récent).

5.2 CONDITIONS D'EXTREMUM

EXTREMUM EN UNE VARIABLE (rappel)

Condition nécessaire du premier ordre (CNO) :

$f'(x^*) = 0 \leftarrow$ point critique (stationnaire)
 Condition suffisante du second ordre (CS0) :
 $f''(x^*) > 0 \rightarrow$ minimum local
 $f''(x^*) < 0 \rightarrow$ maximum local
 $f''(x^*) = 0 \rightarrow$ test insuffisant (inflexion possible)

EXTREMUM EN PLUSIEURS VARIABLES

Condition nécessaire du premier ordre :
 $\nabla f(x^*) = 0 \leftarrow$ gradient nul (point critique)
 C'est-à-dire : $\partial f / \partial x_1 = 0, \partial f / \partial x_2 = 0, \dots, \partial f / \partial x_n = 0$

Condition suffisante du second ordre : étude du Hessian (voir 5.4)

EXTREMUM SOUS CONTRAINTE – MULTIPLICATEURS DE LAGRANGE

Problème : minimiser $f(x)$ sous contrainte $g(x) = 0$

Lagrangien : $L(x, \lambda) = f(x) + \lambda \cdot g(x)$

Conditions de premier ordre (KKT simplifiées) :
 $\nabla_x L = 0 \rightarrow \nabla f(x^*) + \lambda \cdot \nabla g(x^*) = 0$
 $\partial L / \partial \lambda = 0 \rightarrow g(x^*) = 0$

INTERPRÉTATION :
 λ est le "prix" de la contrainte (combien de valeur on gagnerait à relâcher la contrainte d'une unité).

Application data science :
 – Support Vector Machine : maximiser la marge sous contrainte de classification
 – Régression Ridge : minimiser $MSE + \lambda \cdot ||\beta||^2$ (régularisation)
 – Allocation de portefeuille : maximiser rendement sous contrainte de risque

5.3 LE GRADIENT – DIRECTION DE L'OPTIMISATION

DÉFINITION

Le gradient d'une fonction scalaire $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est le vecteur des dérivées partielles :

$$\nabla f(x) = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]^T$$

- Le gradient pointe dans la direction de la plus forte montée de f .
- Son opposé ($-\nabla f$) pointe dans la direction de la plus forte descente.
- Sa norme $||\nabla f(x)||$ mesure la "pente" de f en x .
- Au minimum (ou maximum) : $\nabla f(x^*) = 0$

EXEMPLE – GRADIENT DE LA MSE EN RÉGRESSION LINÉAIRE

$$\begin{aligned}
 L(\beta) &= (1/n) \times ||Y - X\beta||^2 \\
 &= (1/n) \times (Y - X\beta)^T (Y - X\beta)
 \end{aligned}$$

$$\nabla_{\beta} L(\beta) = (-2/n) \times X^T (Y - X\beta)$$

En posant $\nabla_{\beta} L = 0$:
 $X^T (Y - X\beta) = 0$
 $X^T X \beta = X^T Y$
 $\hat{\beta} = (X^T X)^{-1} X^T Y \leftarrow$ estimateur MCO

DESCENTE DE GRADIENT

Algorithme itératif pour minimiser $L(\theta)$:
 $\theta_{k+1} = \theta_k - \eta \cdot \nabla L(\theta_k)$

η (learning rate / pas) : hyperparamètre critique
 – η trop grand $\rightarrow$ oscillations, divergence
 – η trop petit $\rightarrow$ convergence très lente
 – η optimal $\rightarrow$ convergence stable et rapide

VARIANTES DE LA DESCENTE DE GRADIENT

[A] GRADIENT BATCH (GD)
 $\theta_{k+1} = \theta_k - \eta \cdot (1/n) \sum_i \nabla L(\theta_k ; x_i, y_i)$

- Utilise tout le dataset à chaque itération
- Précis mais lent pour n grand

[B] GRADIENT STOCHASTIQUE (SGD)

- $\theta_{k+1} = \theta_k - \eta \cdot \nabla L(\theta_k ; x_i, y_i)$
- Une seule observation par mise à jour
- Rapide, bruyant, bonne généralisation
- Bruit = "régularisation implicite"

[C] MINI-BATCH GRADIENT DESCENT

- $\theta_{k+1} = \theta_k - \eta \cdot (1/|B|) \sum_{i \in B} \nabla L(\theta_k ; x_i, y_i)$
- Compromis : batch de taille B (32, 64, 128...)
- Standard en deep learning

[D] ADAM (Adaptive Moment Estimation)

- $m_t = \beta_1 m_{t-1} + (1-\beta_1) \cdot g_t$ ← moyenne mobile du gradient
- $v_t = \beta_2 v_{t-1} + (1-\beta_2) \cdot g_t^2$ ← moyenne mobile du carré du gradient
- $\hat{m}_t = m_t / (1-\beta_1^t)$ ← correction du biais
- $\hat{v}_t = v_t / (1-\beta_2^t)$
- $\theta_{t+1} = \theta_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$

- $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1e-8$ (valeurs par défaut)
- Adapte le learning rate par dimension
- Converge rapidement → standard en deep learning

[E] RMSProp, Adagrad, AdamW

- Adagrad : accumule les carrés des gradients → décroissance du LR
- RMSProp : version avec fenêtre glissante (corrige la décroissance)
- AdamW : Adam + weight decay décorrélié (meilleure généralisation)

5.4 LA MATRICE HESSIENNE – COURBURE ET SECOND ORDRE

DÉFINITION

Le Hessien d'une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est la matrice des dérivées partielles du second ordre :

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \dots & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Si f est C^2 (deux fois continûment différentiable) → H est symétrique :
 $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$ (théorème de Schwarz)

INTERPRÉTATION GÉOMÉTRIQUE

- H mesure la courbure de la fonction f en un point
- Valeur propre positive → courbure concave vers le haut (cuvette)
- Valeur propre négative → courbure concave vers le bas (colline)
- Valeur propre nulle → direction plate (selle possible)

CONDITIONS DU SECOND ORDRE EN n DIMENSIONS

En un point critique x^* (où $\nabla f(x^*) = 0$) :

$H(x^*)$ définie POSITIVE (toutes valeurs propres > 0)
 → x^* est un minimum local

$H(x^*)$ définie NÉGATIVE (toutes valeurs propres < 0)
 → x^* est un maximum local

$H(x^*)$ INDÉFINIE (valeurs propres de signes mixtes)
 → x^* est un point de selle (saddle point)
 → Ce n'est ni un minimum ni un maximum

$H(x^*)$ semi-définie (au moins une valeur propre $= 0$)
 → Test inconclus → analyse d'ordre supérieur nécessaire

EXEMPLE – HESSIEN DE LA MSE

$$L(\beta) = (1/n) \|Y - X\beta\|^2$$

$$\nabla_{\beta} L = (-2/n) X^T (Y - X\beta)$$

$$H_{\beta} L = (2/n) X^T X$$

$X^T X$ est une matrice de Gram → toujours semi-définie positive.
Si X est de rang plein → $X^T X$ définie positive → L est strictement convexe
→ Le minimum global est unique et c'est l'estimateur MCO.

DÉTERMINANT ET CRITÈRE DE SYLVESTER (cas 2D)

Pour $f(x, y)$ en un point critique (x^*, y^*) :

$$H = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{bmatrix}$$

$$D = \det(H) = f_{xx} \cdot f_{yy} - (f_{xy})^2$$

Si $D > 0$ et $f_{xx} > 0$ → minimum local
Si $D > 0$ et $f_{xx} < 0$ → maximum local
Si $D < 0$ → point de selle
Si $D = 0$ → test inconclus

5.5 MÉTHODE DE NEWTON – OPTIMISATION DU SECOND ORDRE

PRINCIPE

Utiliser le gradient ET le Hessien pour converger plus rapidement :

$$\theta_{k+1} = \theta_k - H(\theta_k)^{-1} \cdot \nabla L(\theta_k)$$

INTERPRÉTATION :

On approche $L(\theta)$ par une parabole (développement de Taylor d'ordre 2) :
 $L(\theta) \approx L(\theta_k) + \nabla L(\theta_k)^T (\theta - \theta_k) + (1/2) (\theta - \theta_k)^T H(\theta_k) (\theta - \theta_k)$

En minimisant cette approximation quadratique :
→ $\theta^* = \theta_k - H(\theta_k)^{-1} \nabla L(\theta_k)$

AVANTAGES ET INCONVÉNIENTS

AVANTAGES :

- Convergence quadratique (très rapide près du minimum)
- Pas de choix de learning rate
- Prend en compte la courbure → pas "trop grands" ou "trop petits"

INCONVÉNIENTS :

- Calcul de H et de H^{-1} : coût $O(n^3)$ pour n paramètres
- Inadapté pour les grands réseaux de neurones (milliards de paramètres)
- H peut ne pas être définie positive → divergence possible

MÉTHODES QUASI-NEWTON (APPROXIMATIONS DU HESSIEN)

BFGS (Broyden–Fletcher–Goldfarb–Shanno) :

- Approximer H^{-1} itérativement sans la calculer exactement
- Mise à jour : $H_{k+1}^{-1} = f(H_k^{-1}, \text{gradient}, \text{direction})$

L-BFGS (Limited-memory BFGS) :

- Ne stocker que les m dernières mises à jour ($m \approx 10-20$)
- Coût mémoire $O(m \cdot n)$ au lieu de $O(n^2)$
- Standard pour optimisation de modèles ML classiques
- Utilisé dans scikit-learn pour régression logistique, SVM

5.6 EXTREMUM GLOBAL – MÉTHODES AVANCÉES

RECUIT SIMULÉ (Simulated Annealing)

Inspiré du recuit des métaux en physique.
Accepte parfois des solutions moins bonnes (contrôlé par la "température").
→ Évite les minima locaux sur des espaces non-convexes.

ALGORITHMES GÉNÉTIQUES

Population de solutions candidates.
Sélection, croisement, mutation.
→ Optimisation globale heuristique.
Usage : feature selection, hyperparameter tuning.

OPTIMISATION BAYÉSIENNE (Bayesian Optimization)

Construire un modèle probabiliste de la fonction objectif
(Gaussian Process) puis choisir intelligemment le prochain point à évaluer.

- Très efficace quand chaque évaluation est coûteuse (entraînement ML).
- Standard pour l'hyperparameter tuning (Optuna, Hyperopt, Ray Tune).

Processus :

- [1] Placer un prior gaussien sur $L(\theta)$
- [2] Observer $L(\theta)$ en quelques points
- [3] Calculer le posterior (GP mis à jour)
- [4] Choisir le prochain θ via une acquisition function (EI, UCB, PI)
- [5] Répéter jusqu'à convergence

5.7 RÉGULARISATION VUE COMME OPTIMISATION CONTRAINTE

RÉGRESSION RIDGE (L2)

Minimiser : $\|Y - X\beta\|^2 + \lambda \|\beta\|^2$

Solution : $\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T Y$

- λI rend la matrice inversible même en cas de multicollinéarité
- Shrinkage : les coefficients sont contractés vers 0
- $\lambda \rightarrow \infty$: $\beta \rightarrow 0$ (sous-ajustement)
- $\lambda \rightarrow 0$: $\beta \rightarrow \hat{\beta}_{\text{MCO}}$ (sans régularisation)

RÉGRESSION LASSO (L1)

Minimiser : $\|Y - X\beta\|^2 + \lambda \|\beta\|_1$

- Produit des solutions parcimonieuses (certains $\beta_i =$ exactement 0)
- Sélection automatique de variables
- Pas de solution analytique → descente de gradient ou LARS

ELASTIC NET (L1 + L2)

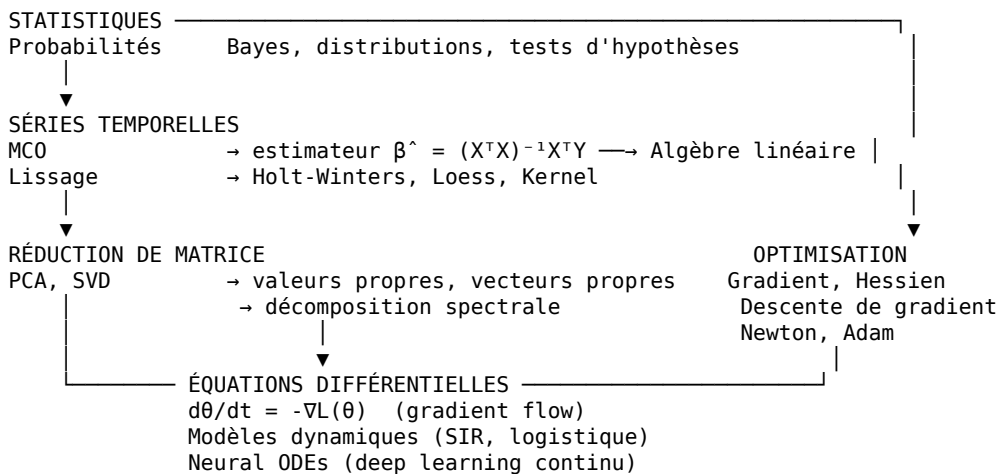
Minimiser : $\|Y - X\beta\|^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|^2$

- Combine les avantages de Ridge et Lasso

INTERPRÉTATION DU PARAMÈTRE λ (HESSIEN) :

- Ridge ajoute λI au Hessien $X^T X$ → rend H définie positive même quand $X^T X$ est mal conditionnée.
- La régularisation L2 "conditionne" le problème d'optimisation.

SYNTHÈSE – CARTE MENTALE DES CONNEXIONS



RÈGLE D'OR GCI :

- "Un data scientist senior ne choisit pas entre statistiques et ML.
- Il comprend que le ML est de la statistique appliquée à grande échelle,
- et que l'optimisation est le moteur caché derrière chaque modèle."

RÉFÉRENCES ET OUTILS PYTHON PAR THÈME

STATISTIQUES & PROBABILITÉS

- scipy.stats – tests d'hypothèses, distributions, intervalles de confiance
- statsmodels – régression, tests, analyse statistique complète

pingouin – tests statistiques avec sorties lisibles

SÉRIES TEMPORELLES

statsmodels.tsa – ARIMA, SARIMA, Holt-Winters, tests ADF
prophet (Meta) – prévision avec tendance + saisonnalité
sktime – framework unifié séries temporelles ML
tslearn – clustering et classification de séries temporelles

ALGÈBRE LINÉAIRE & RÉDUCTION

numpy.linalg – valeurs propres, SVD, inverse, rang
sklearn.decomposition – PCA, NMF, FastICA, TruncatedSVD
sklearn.manifold – t-SNE, UMAP (via umap-learn)

ÉQUATIONS DIFFÉRENTIELLES

scipy.integrate – solve_ivp, odeint
torchdiffeq – Neural ODEs avec PyTorch

OPTIMISATION

scipy.optimize – minimize, linprog, curve_fit, BFGS, L-BFGS-B
cvxpy – optimisation convexe déclarative
optuna – optimisation bayésienne hyperparamètres
torch.optim – SGD, Adam, AdamW, RMSProp, LBFGS

=====

GCI – GYKHAMINE CONCEPT INVESTIGATION

"La donnée est brute. La mathématique la sculpte. Le sens vous appartient."

=====